KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY

Bhubaneswar, Odisha

AI SHOPPING HACKATHON 2026

AI Shopping Agent

Conversational Commerce Across Multiple Categories

React · Node.js · Shopify API · AI Recommendations · Voice Assistant

TEAM

Aarushi Sharma · MD. Khustar Noorani

github.com/aarushhii30/AI-shopping-agent

Submitted for: Shopify AI Shopping Hackathon 2026

Category: Conversational Commerce / AI Shopping Assistant

1 PRODUCT OVERVIEW

We built a conversational AI shopping assistant that spans multiple product categories — fashion, footwear, gadgets, accessories, and home decor. Embedded inside a full shopping flow, the assistant supports natural-language discovery, AI-powered recommendations, smart product search, cart management, and multi-step checkout. The backend integrates with the Shopify API for live product data, while a voice input layer allows hands-free querying.

The core premise: instead of a user scrolling through category trees and filters, they should be able to describe what they want — and get relevant results instantly.

2 PROBLEM STATEMENT

Most e-commerce discovery experiences are broken for users who don't know exactly what they want. Traditional search requires precise keyword input. Filter-based browsing assumes the user understands the product taxonomy. Both fail when a user comes with vague intent: "I need a gift for my mom" or "something for the gym under ₹500."

Existing AI shopping tools either live inside a single category (one brand's recommendation engine) or are too generic (a chatbot linking to Google). There is no unified conversational layer that works across multiple product types in a real shopping environment with actual cart and checkout integration.

3 WHY THIS PROBLEM MATTERS

Discovery friction directly hurts conversion. A user who cannot find what they want within a few interactions either bounces or defaults to a competitor. For smaller Shopify merchants — who lack budget for personalization engines — the gap is even wider. A conversational AI layer that plugs into existing catalogs is a low-effort, high-impact tool for these merchants.

Voice- and chat-driven interfaces are increasingly the norm on mobile. Shopping experiences built purely for desktop keyword search are losing relevance. Building for conversational commerce now is a forward-looking bet, not a stretch goal.

4 TARGET USERS & PAIN POINTS

▸ Buyers (End Consumers)

— Juggling multiple tabs or stores to find a product matching a vague need

- Filter navigation is confusing when they don't know product terminology
- Cannot easily cross-category shop (e.g., gym bag + water bottle in one query)
- On mobile, typing long search queries is friction — voice is more natural

▸ Merchants (Shopify Store Owners)

- No affordable plug-and-play AI discovery layer for their existing catalog
- Losing customers at the discovery stage before they reach the product page
- No easy way to surface lesser-known items through traditional search

5 WHAT WE DECIDED TO BUILD

We chose to build a conversational discovery and checkout layer on top of the Shopify catalog. The decision to go multi-category was intentional — we wanted to validate whether a single AI interface could handle meaningfully different shopping behaviors (spec-comparison for gadgets vs. aesthetic preference for fashion vs. visual browsing for home decor) without requiring category-specific tuning.

▸ Committed Scope for the Hackathon

- User login and signup via Firebase Authentication (email/password, JWT session persistence)
- Natural language query → AI-interpreted product recommendations
- Voice input support for hands-free querying
- Smart product search with AI-ranked results from Shopify catalog
- Full cart flow: quantity management, promo codes, GST calculation
- Multi-step checkout: Shipping → Payment → Review
- Suggested prompt chips to reduce the blank-page problem for new users

6 CORE USER JOURNEY

Step 1 — Entry

User lands on the auth screen and signs up or logs in via Firebase. Once authenticated, they are redirected to the AI chat interface where they type a natural language query or tap a suggested prompt (e.g., “Show me bags under ₹500”, “Best products for the gym”). Login state is persisted via JWT so returning users skip the auth step entirely.

Step 2 — Discovery

The AI interprets intent, queries the Shopify catalog, and returns ranked product cards with image, name, price, and a short AI-generated match reason. Users scroll results without leaving the chat view.

Step 3 — Add to Cart

Users add items directly from the recommendation view. The cart panel shows quantity controls, a promo code input, and an order summary with subtotal, shipping, and GST breakdown.

Step 4 — Checkout

A 3-step checkout flow: shipping details, payment method selection (Card, UPI, PayPal, Apple Pay, Google Pay), and order review before placing.

7 KEY PRODUCT DECISIONS & REASONING

▸ Multi-category over single niche

We could have gone deeper on one category (e.g., a pure fashion assistant with style profiling). Instead, we chose breadth to test whether a generalized conversational layer could hold across different shopping behaviors. The tradeoff: category-specific features like "match this outfit" or "compare laptop specs" were out of scope — but we proved the flexible recommendation engine concept first.

▸ Shopify API as the product backbone

Rather than using a mock dataset, we integrated with the real Shopify API. This made the product feel honest — recommendations pull from an actual catalog, not a curated JSON file. The downside was rate limits and occasional latency in search responses.

▸ Suggested prompt chips

We added clickable prompt suggestions after early testing revealed that a blank chat input created hesitation. The chips reduce the blank-page problem and guide users toward productive queries without making the interaction feel scripted.

▸ Voice input integration

Voice was added because the mobile UX case was strong — typing detailed queries on mobile is cumbersome. The voice layer transcribes input and feeds it through the same AI pipeline as typed queries, keeping the implementation lean. We did not build wake-word detection or persistent voice sessions.

▸ Separate cart and checkout screens

Checkout was built as distinct views rather than a sidebar or modal to give it the visual weight it deserves. Users tend to trust checkout pages that look like real pages, not overlays. The tradeoff is a slightly longer navigation path, but it felt more trustworthy.

8 FEATURES INTENTIONALLY NOT BUILT

▸ Virtual Try-On

Obvious value for fashion — skipped because it requires a CV pipeline (AR/ML) or a third-party API (Snap, Zeekit), both of which would have taken 3–4x the hackathon timeline. A half-baked try-on would hurt credibility more than help.

▸ AI Product Comparison

Comparing two products side-by-side via conversation is high-value for gadgets. Deprioritized because it requires structured attribute extraction per product category — a schema we didn't have time to design well.

▸ Conversational Checkout

Ideally, a user could say "checkout with my saved address and Google Pay" and have it happen. We built a traditional checkout UI instead — the conversational state management (tracking shipping/payment choices mid-chat) required testing cycles we didn't have.

▸ Autonomous Shopping Actions

Autonomous reorders, price-drop alerts, and completing purchases on behalf of the user require authenticated sessions, persistent profiles, and trust mechanisms. These are product problems that need more than a hackathon weekend.

▸ User Accounts & Order History

We implemented login and signup flows using Firebase Authentication. Users can register with email/password or sign in to an existing account, with session state managed via JWT tokens through AuthContext. This ensures the shopping session is tied to an authenticated user, laying the groundwork for persistent order history and personalisation in future iterations. Full order history and saved preferences are scoped to the next phase.

9 TRADEOFFS & HOW WE HANDLED THEM

▸ Breadth vs. depth of category intelligence

Going multi-category meant we couldn't fine-tune AI prompting per category. A fashion query and a gadgets query go through the same interpretation layer. We compensated with a flexible system prompt that encourages the AI to identify category context from the query itself before matching products.

▸ Real API vs. reliability

Shopify API calls add latency — some queries took 3–5 seconds under load. We added a loading indicator and typed-text animation to mask the wait perceptually. The long-term fix is caching popular queries and pre-fetching category-level results.

▸ Voice UX vs. cross-browser support

Web Speech API is inconsistent across browsers. We scoped voice to Chrome/Edge only and surfaced a clear error state on unsupported browsers rather than silently degrading. Not ideal, but honest.

▸ Checkout completeness vs. scope

The checkout is a UI prototype — no real payment processing. Building real payment (Stripe or Razorpay) would have shifted focus off the AI shopping layer, which is the actual innovation. We were transparent about this in the demo context.

10 FUTURE IMPROVEMENTS & SCALABILITY

If we were to continue building post-hackathon, the priority roadmap would be:

- Category-specific AI tuning — separate prompt strategies for fashion (aesthetic), gadgets (spec comparison), and home decor (visual affinity)
- Persistent user profiles — login/signup is live via Firebase; next step is saving preferences, past searches, and order history for true personalisation
- Conversational checkout — payment and address confirmation via chat, not a form
- AI product comparison — side-by-side structured comparison for spec-heavy categories
- Virtual try-on — AR integration for fashion and accessories via a third-party CV API
- Autonomous reorder actions — allow trusted users to let the AI reorder consumables
- Merchant dashboard — analytics on what users are searching for but not finding
- Multi-language support — Hindi, Tamil, Bengali and other Indian languages

The current build is a foundation — it proves that conversational commerce works and checkout integrates cleanly. The next phase is going deeper on intelligence, not wider on features.